

Alocação de Endereços IPv6 em Redes Multi-hop de Rádios de Baixa Potência

Bruna Soares Peres¹, Olga Goussevskaja¹

¹Departamento de Ciência da Computação – Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

{bperes, olga}@dcc.ufmg.br

Abstract. *In this work, we propose a Multi-hop Host Configuration strategy that explores cycle-free network structures in low-power wireless networks to generate and assign IPv6 addresses to nodes. The objective is to enable efficient and robust top-down routing with low memory footprint. MHCL generates and assigns IPv6 addresses which reflect the topology of the underlying wireless network. We implemented our strategy as a subroutine of RPL protocol in Contiki OS. We performed experiments to show that our strategy is efficient in time and number of messages, and is robust to network dynamics, caused by failures in network links and nodes.*

Resumo. *Muitas redes sem fio de baixa potência são baseadas em mecanismos que mantêm topologias acíclicas para suportar aplicações de coleta de dados, tipicamente otimizada para o tráfego ascendente de dados. Pacotes que precisam ser enviados em uma direção diferente devem seguir por caminhos mais longos e são frequentemente descartados devido à falta de memória para as tabelas de roteamento. Neste trabalho propomos uma estratégia de configuração de nós que explora estruturas acíclicas em redes sem fio de baixa potência para gerar e atribuir endereços IPv6 para os nós. O objetivo é permitir o roteamento descendente de maneira eficiente e robusta, com baixo consumo de memória. Isso é alcançado ao gerar e atribuir endereços IPv6 que refletem a topologia da rede sem fio. Nós implementamos a estratégia proposta como uma rotina do protocolo RPL no sistema operacional Contiki. Foram realizados experimentos para demonstrar que a estratégia é eficiente em termos de tempo e número de mensagens, além de ser robusta à dinâmica da rede, decorrente de falhas em enlaces e nós da rede.*

1. Introdução

A principal função de uma rede sem fio de baixa potência é, tipicamente, um tipo de coleta de dados [Liu et al. 2013, Levis et al. 2008]. Aplicações baseadas na coleta de dados são muitas e os exemplos incluem monitoramento do meio ambiente [Tolle et al. 2005], segurança [Vicaire et al. 2009] e observações científicas [Werner-Allen et al. 2006]. De modo a realizar a coleta de dados, uma estrutura de grafo acíclico é tipicamente mantida e um *convergecast* (fluxo de dados de baixo para cima a partir das folhas em relação à raiz) é implementado nessa topologia de rede. Muitos sistemas operacionais para nós sensores (por exemplo, Tiny OS [Levis et al. 2005] e *Contiki OS* [Dunkels et al. 2004,

con J) implementam mecanismos para manter topologias de rede acíclicas para suportar aplicações de coleta de dados.

Em algumas situações, entretanto, o fluxo de dados na direção oposta torna-se necessário. Um algoritmo de coleta de dados muito comum em redes sem fio, conhecido como *convergecast*, pode ativar o tráfego de dados no sentido descendente em cenários onde falhas são comuns, por exemplo. O algoritmo de *convergecast* pode ser formalmente definido como se segue: em um ambiente consistindo de nós de sensores sem fios, cada nó i possui uma unidade de dados que deve ser transmitida para a estação base. Cada nó possui um alcance de transmissão, que pode ser ajustado até um valor máximo de r . Todos os nós dentro da área de transmissão são considerados os seus vizinhos. Um nó sensor, se dentro da faixa, pode transmitir seus dados para a estação de base diretamente (*singlehop*) ou através de outro nó (*multihop*) [Upadhyayula et al. 2003]. Quando a rede opera em um cenário onde falhas são comuns, é possível que os nós optem por receber uma confirmação da raiz sobre o recebimento de suas mensagens, ou seja, a raiz ou roteador de borda deve enviar uma mensagem descendente para um determinado nó, assim que receber uma mensagem de coleta desse nó.

Mesmo o tráfego de dados de cima para baixo não sendo tipicamente o principal objetivo das redes sem fio *multi-hop* de baixa potência, ele é implementado por alguns esquemas de roteamento populares como uma função “extra”, o que significa que as rotas são otimizadas para o tráfego de baixo para cima e mensagens descendentes podem seguir por rotas mais longas ou não serem entregues devido a restrições de memória. No protocolo RPL, por exemplo, é mantida uma topologia de rede acíclica, chamada DAG (*Directed Acyclic Graph*), em que cada nó mantém na memória uma pequena lista de nós pais, ordenados de acordo com a função objetivo, para os quais ele encaminha os dados em direção ascendente à raiz. Se um nó quer atuar como destino, ele envia uma mensagem para cima através do DAG, e os nós intermediários (se eles operam no assim chamado modo *storing*) adicionam uma entrada para este destino na tabela de roteamento criada especificamente para rotas descendentes. O tamanho de cada tabela deste tipo é potencialmente $O(n)$, onde n é o tamanho da sub-árvore com raiz em cada nó, ou seja, o número total de descendentes do nó. Dado que a capacidade de memória pode ser altamente restringida em redes sem fio de baixa potência, muitas vezes nem todas as rotas podem ser armazenadas e, conseqüentemente, pacotes serão descartados. Isto resulta na perda de um número elevado de mensagens e um alto consumo de memória. A fim de reduzir o tamanho da tabela de roteamento, seria desejável agregar vários endereços de nós de destinos próximos em uma única entrada da tabela.

Neste trabalho, propomos o MHCL (*IPv6 Multi-hop Host Configuration for Low-power wireless networks*)—uma estratégia de configuração de nós *multi-hop* que explora estruturas acíclicas em redes sem fio de baixa potência para gerar e atribuir endereços IPv6 para os nós. O objetivo é permitir tráfego descendente eficiente e robusto (broadcast, multicast e unicast) com baixo consumo de memória, ou seja, tabelas de roteamento pequenas. Propomos e analisamos duas estratégias de alocação de endereços: *top-down* e *bottom-up*. Na abordagem *top-down*, cada nó realiza um particionamento do espaço de endereços disponível em intervalos de endereços de tamanhos iguais e distribui essas fatias a seus filhos. A abordagem *bottom-up* contém uma fase inicial de agregação, em que nós computam seu número de descendentes. Uma vez que a raiz recebe o número

total de descendentes de cada um dos seus filhos (imediatos), atribui uma fatia/partição de endereços proporcional ao tamanho da sub-árvore com raiz em cada um de seus filhos. Uma vez que um nó recebe uma fatia de endereço de seu pai preferencial, ele particiona essa faixa entre seus próprios filhos, usando o mesmo algoritmo.

O MHCL gera e atribui endereços IPv6 que refletem a topologia da rede sem fio subjacente. Como resultado, as tabelas de roteamento para tráfego descendente são de tamanho $O(k)$, onde k é o número de filhos (imediatos) de cada nó de roteamento em uma topologia acíclica. Foram usados temporizadores inspirados no algoritmo *Trickle* [Levis et al. 2004] para ajustar o mecanismo de comunicação em nosso protocolo, de modo que ele é capaz de se adaptar rapidamente à dinâmica na topologia da rede, sem inundar a rede com mensagens de controle. Analisamos quão robusta é nossa estratégia em relação à dinâmica da rede, decorrente de falhas em enlaces e nós. Mostramos que, se os endereços são atribuídos na fase de configuração inicial da rede, as mensagens podem ser entregues com alta taxa de sucesso. Demonstramos por meio de simulações (usando o simulador *Cooja* [Eriksson et al. 2009] do sistema operacional *Contiki OS* [Dunkels et al. 2004] com implementação do protocolo RPL) que o MHCL é robusto à dinâmica da topologia de redes sem fio de baixa potência.

O restante deste artigo está organizado da seguinte forma. Na Seção 2, nós descrevemos brevemente como o tráfego descendente é implementado no protocolo RPL. Na Seção 3 apresentamos as versões top-down e bottom-up do MHCL. Na Seção 4 apresentamos nossos resultados experimentais. Na Seção 5 discutimos trabalhos relacionados e, na Seção 6 encerramos com algumas conclusões finais.

2. Sistemas 6LoWPAN

Sistemas 6LoWPAN (*IPv6 over Low power Wireless Personal Area Networks*¹) são compostos de dispositivos de baixo custo que permitem comunicação sem fio, porém com recursos limitados, obedecendo ao padrão IEEE802.15.4. Nessas redes, perdas de pacotes são extremamente frequentes, e os links podem se tornar inutilizáveis por algum tempo [Vasseur and Dunkels 2010].

2.1. Endereçamento IPv6

Entre a rede (global) IPv6 e uma rede 6LoWPAN existe um roteador de borda que realiza a compressão do cabeçalho IPv6. Os 128 bits do endereço IPv6 são normalmente divididos em duas partes: o prefixo da rede (64 bits) e o endereço do nó (64 bits). O mecanismo de compressão de cabeçalho 6LoWPAN omite os bits do prefixo de rede, pois eles são fixos para uma determinada rede 6LoWPAN. Uma vez que os outros 64 bits podem abordar um espaço de endereçamento muito grande, o 6LoWPAN fornece opções para comprimir o endereço do nó (o uso comum é de 16 bits)². Similarmente ao IPv4, existem duas maneiras de configuração de endereços: dinâmica ou estática. A atribuição de endereço estático precisa ser configurada manualmente em cada dispositivo. A configuração dinâmica é realizada pelo protocolo DHCPv6 (*Dynamic Host Configuration Protocol version 6*). Dado que a rede é *multihop*, isso requer a execução de um servidor DHCPv6 e vários agentes retransmissores na rede.

¹<http://datatracker.ietf.org/doc/rfc4919>

²<http://datatracker.ietf.org/doc/rfc6282>

2.2. Protocolo RPL

O RPL [Winter et al. 2012] é um protocolo de roteamento especificamente projetado para redes 6LoWPAN. A base do protocolo é construção e manutenção de uma topologia acíclica de rede direcionada à raiz. Para isto, um Grafo Acíclico Direcionado Orientado ao Destino (DODAG) é criado. Para iniciar o processo de construção do DAG a raiz anuncia informações sobre o grafo por meio de mensagens do tipo DIO (*DODAG Information Object*). Uma vez que um nó se junta ao DAG, ele armazena uma lista de potenciais pais, ordenada pela qualidade das respectivas conexões, e marca o primeiro pai da lista como preferencial. Em seguida, o nó calcula seu *rank*, uma métrica que indica as coordenadas do nó na hierarquia da rede [Vasseur et al. 2011]. Quando esse processo se estabiliza, a coleta de dados pode começar.

RPL especifica dois modos de operação para o roteamento descendente: *storing* e *non-storing*. Ambos os modos requerem a geração de mensagens do tipo DAO (*Destination Advertisement Object*) pelos nós que desejam atuar como destinos, ou seja, receber mensagens endereçadas a eles. No modo de operação *storing*, cada roteador RPL devem armazenar rotas para seus destinos em um DODAG. Essas informações são repassadas dos nós para seus pais preferenciais. Dessa forma, a memória necessária em um nó próximo à raiz é $O(n)$, onde n é o tamanho da sub-árvore com raiz em cada nó, ou seja, o número total de descendentes do nó. Dado que a capacidade de memória pode ser altamente restringida em redes sem fio de baixa potência, muitas vezes nem todas as rotas podem ser armazenadas e, consequentemente, pacotes serão descartados. Isso resulta na perda de um número elevado de mensagens e um alto consumo de memória. No modo *non-storing*, os nós não possuem capacidade de armazenamento e as mensagens DAO são enviadas diretamente para a raiz. A raiz é o único nó que armazena a tabela de roteamento com as rotas descendentes. Para enviar mensagens para os nós, é então utilizado o roteamento de origem (*source routing*), ou seja, as rotas são enviadas dentro dos próprios pacotes.

2.3. Trickle

O algoritmo *Trickle* permite que nós em um meio compartilhado com perdas possam trocar informações de uma maneira altamente robusta, simples e escalável, com um baixo consumo de energia [P. Levis 2011]. Ao ajustar as janelas de transmissão dinamicamente, o Trickle é capaz de difundir novas informações sobre a escala de tempo de transmissão da camada de enlace enquanto envia apenas algumas mensagens por hora quando a informação não se altera. No algoritmo *Trickle*, esporadicamente um nó transmite dados sobre a topologia, a não ser que ele escute alguma outra transmissão com dados que sugiram que sua própria transmissão é redundante [Levis et al. 2004]. Existem dois resultados possíveis de uma mensagem do Trickle. Cada nó que escuta a mensagem pode perceber que os dados da mensagem são consistentes com seu próprio estado ou um receptor pode detectar uma inconsistência. Dessa forma, o algoritmo Trickle pode ser utilizado para o estabelecimento de eventual consistência de uma rede sem fio e para otimizar a disseminação de informações sobre a topologia, como é o caso de sua utilização no protocolo RPL [Winter et al. 2012]. No MHCL, o Trickle foi utilizado como base para a definição de um temporizador responsável pela estabilização da rede. Quando uma informação sobre a topologia não se altera, o temporizador tem seu valor dobrado até atingir o máximo. Caso a informação se altere, o valor é redefinido ao inicial e o nó

deve esperar até o temporizador chegar ao valor máximo para ter certeza que a topologia está estável.

3. MHCL: Alocação de Endereços IPv6 em Redes Multi-hop de Rádios de Baixa Potência

O objetivo do MHCL (*IPv6 Multi-hop Host Configuration in Low-power wireless networks*) é implementar um esquema de alocação de endereços IPv6 para tráfego descendentes que consuma pouca memória, ou seja, necessite de pequenas tabelas de roteamento. O espaço de endereços disponível do roteador de borda, os 64 bits menos significativos do endereço IPv6, é particionado de maneira hierárquica entre os nós conectados por meio de uma topologia *multihop* acíclica. Cada nó recebe uma faixa de endereços IPv6 de seu pai e particiona essa faixa entre seus filhos, até que todos os nós tenham sido endereçados. Uma vez que a alocação de endereços é realizada de maneira hierárquica, a tabela de roteamento de cada nó deve ter k entradas, onde k é o número de filhos (diretos). Cada entrada da tabela de roteamento agrega o endereço de todos os nós na subárvore com raiz no nó filho correspondente.

Para decidir como o espaço de endereçamento disponível deve ser particionado, os nós precisam coletar informações sobre a topologia da rede. Uma vez que uma versão *estável* da topologia da rede é alcançada, a raiz começa a distribuir as fatias de endereços. Note que o conceito de estabilidade é importante para implementar um particionamento de endereços coerente. Portanto, o MHCL possui uma fase de inicialização, durante a qual informações sobre a topologia são atualizadas progressivamente, até que um período de tempo (pré-definido) suficientemente longo se passe sem alterações na topologia. Note que, uma vez que a rede atinge um estado de estabilidade inicial, mudanças de topologia futuras são esperadas ser de natureza local, provocadas por uma conexão falha ou falha de um nó. No (atípico) caso de mudanças globais e permanentes na topologia da rede, o algoritmo MHCL deve ser reiniciado pela raiz.

No restante desta seção nós descrevemos os principais componentes do MHCL: a alocação de endereços IPv6 (Seção 3.1), os tipos de mensagens (Seção 3.2), as rotinas de comunicação e temporizadores (Seção 3.3) e as tabelas de roteamento e rotinas de encaminhamento (Seção 3.4).

3.1. Alocação de Endereços IPv6

Nós propomos duas estratégias de particionamento do espaço de endereçamento: top-down e bottom-up. Na abordagem top-down (Figura 1(a)), cada nó conta o seu número k de filhos, espera até que seu pai preferencial lhe alocue uma faixa de endereços (caso o nó não seja raiz), particiona o seu espaço de endereçamento em k faixas de igual tamanho, reservando uma parte de $r\%$ do total para possíveis conexões futuras, e então distribui as faixas resultantes para os filhos. Note que esta abordagem utiliza apenas informação sobre nós a uma distância de um *hop*, o que lhe proporciona uma fase relativamente rápida de inicialização.

Na abordagem bottom-up (Figura 1(b)), cada nó contabiliza, através de uma etapa de agregação, o número total de seus descendentes e o repassa para o seu pai preferencial. Além disso, cada nó mantém o número de descendentes de cada filho. Uma vez que as informações se estabilizam, a raiz inicia o processo de distribuição de endereços. Para

isso, ela particiona o espaço de endereçamento disponível entre os seus filhos, de forma que cada um receba uma faixa de tamanho proporcional ao seu respectivo número de descendentes, deixando novamente uma faixa de endereços de reserva. Cada nó, ao receber a sua faixa do pai, repete o procedimento de particionamento proporcional ao número de descendentes de cada filho, até que todos os nós sejam endereçados. A ideia é alocar faixas de endereços maiores para subárvores maiores, de forma a maximizar o aproveitamento do espaço de endereçamento. Isso se torna especialmente relevante em redes muito grandes. Note que esta abordagem utiliza informação agregada ao longo de vários *hops*, o que torna a sua fase de inicialização relativamente mais longa.

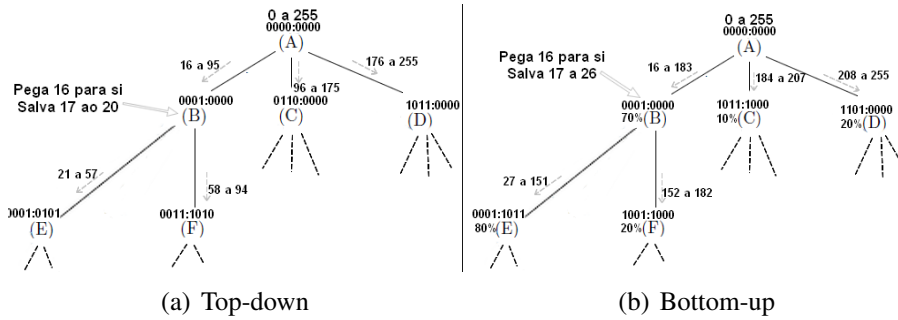


Figura 1. Exemplos de particionamento do espaço de endereçamento.

3.2. Mensagens

MHCL utiliza quatro tipos de mensagens, implementadas modificando-se as mensagens *DIO* e *DAO* do RPL: DIO_{MHCL} , $DIOACK_{MHCL}$, DAO_{MHCL} e $DAOACK_{MHCL}$. Mensagens do tipo DIO_{MHCL} (Figura 2(a)) são enviadas em rotas descendentes, ou seja, dos pais para os filhos. Essa mensagem é utilizada durante a fase do endereçamento, pois ela carrega as informações necessárias para um nó definir seu endereço e a faixa recebida de seu pai. O campo *flag* da mensagem *DIO* do RPL é definido em 1 e o endereço do nó e tamanho da faixa de endereços são enviados no campo *options*. Mensagens do tipo $DIOACK_{MHCL}$ são identificadas por *flag* = 2 e usadas para confirmar o recebimento de mensagens DIO_{MHCL} . Mensagens do tipo DAO_{MHCL} (Figura 2(b)) são enviadas em sentido ascendente, de filho para pai, e possuem duas funções: na abordagem bottom-up, elas carregam o número de descendentes do nó e na abordagem top-down, servem para informar ao pai que ele foi escolhido como preferencial pelo respectivo filho. Essas mensagens são confirmadas por mensagens do tipo $DAOACK_{MHCL}$.

3.3. Comunicação e Temporizadores

O algoritmo MHCL top-down começa com a rotina de *informar pai preferencial*. Se um nó não é raiz, ele deve informar a seu pai que ele é um de seus filhos, enviando uma mensagem DAO_{MHCL} . Um nó deve esperar sua lista de potenciais pais se estabilizar e só então informar ao pai escolhido. No Algoritmo 1, duas constantes são usadas: DIO_{min} é um parâmetro de temporização usado pelo RPL para que um nó defina o menor intervalo para enviar mensagens *DIO*. $peFilho$ é um parâmetro de estabilização usado para decidir quando o pai preferido está estabilizado (a Tabela 1 contém os valores desses parâmetros usados nas simulações). Uma vez que a escolha do pai está estável, a variável local *definiuPai* é definida em *TRUE* e uma mensagem DAO_{MHCL} é enviada para o pai

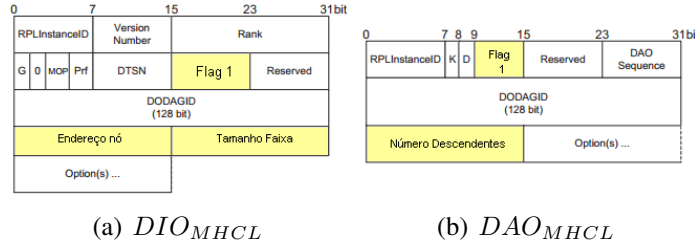


Figura 2. Tipos de mensagens usados no MHCL.

para informar sobre a decisão (linhas 12–18, Algoritmo 1). Em seguida, cada nó ao receber mensagens (DAO_{MHCL}) de seus filhos, salvar uma lista com os filhos³, atualiza o contador de filhos e confirma o recebimento do pacote para cada filho, enviando um $DAOACK_{MHCL}$. O processo de contagem dura até a informação se tornar estável, o que é controlado pelo parâmetro $pePai$. Uma vez que o número de filhos se torna estável, a variável local $definiuFilhos$ se torna verdadeira (linha 13, Algoritmo 2).

Na fase de endereçamento, se um nó não é raiz, ele deve esperar até receber uma faixa de endereços alocada a ele por seu pai (em uma mensagem DIO_{MHCL}) e confirmar o recebimento enviando uma mensagem $DIOACK_{MHCL}$. O particionamento e alocação das faixas de endereços é iniciado pela raiz assim que ela determina o número de filhos que possui ($definiuFilhos$ é verdadeiro). A faixa de endereços disponível é particionada em porções iguais entre os filhos conhecidos, guardando uma reserva para futuros filhos, ou conexões atrasadas. Uma vez que o endereço é particionado, a fatia correspondente a cada filho é enviada em uma mensagem DIO_{MHCL} , como mostra o Algoritmo 5. Se uma mensagem DAO_{MHCL} é recebida de um novo filho após a etapa de distribuição de endereços ter sido completada, então o processo de partição e alocação de endereços é repetido, usando a faixa reserva (a faixa reserva é a quantidade de endereços que o nó possui após realizar o endereçamento de seus filhos).

```

1: definiuPai = FALSE;
2: maxTimer =  $peFilho * DIO_{min}$ ;
3: timer = rand( $1/2 * DIO_{min}, DIO_{min}$ );
4: while NÃO definiuPai do
5:   if NÃO-RAIZ and ESTOUROU TIMER then
6:     if MUDOU-PAI then
7:       reseta timer;
8:     else
9:       if timer < maxTimer then
10:        timer *= 2;
11:       else
12:        definiuPai = TRUE;
13:        if MHCL-top-down then
14:          envia  $DAO_{MHCL}$  para pai;
15:          if NÃO  $DAOACK_{MHCL}$  then
16:            envia  $DAO_{MHCL}$  para pai;
17:          end if
18:        end if
19:      end if
20:    end if
21:  end if
22: end while

```

Algoritmo 1: MHCL: Timer Pai Preferido

```

1: definiuFilhos = FALSE;
2: maxTimer =  $pePai * DIO_{min}$ ;
3: timer = rand( $1/2 * DIO_{min}, DIO_{min}$ );
4: count = 0;
5: while NÃO definiuFilhos do
6:   if ESTOUROU-TIMER then
7:     if MUDOU-CONTADOR then
8:       reseta timer;
9:     else
10:      if timer < maxTimer then
11:        timer *= 2;
12:      else
13:        definiuFilhos = TRUE;
14:        end if
15:      end if
16:    end if
17:  end while

```

Algoritmo 2: MHCL top-down: Timer Contador Filhos

³Note que na implementação padrão de estruturas acíclicas em protocolos como RPL e CTP, um nó não salva uma lista de filhos, apenas uma lista de potenciais pais

A abordagem Bottom-Up do MHCL primeiramente executa um procedimento de agregação para computar o número total de descendentes de cada nó. Se um nó não é a raiz e ele já definiu seu pai preferido (*definiuPai* é verdadeiro) ele começa enviando uma mensagem DAO_{MHCL} com o contador $count = 0$ (Algoritmo 3). O nó então, após enviar seu primeiro DAO_{MHCL} espera por mensagens desse tipo de seus filhos, atualizando o número de descendentes a cada mensagem recebida e controlando o temporizador de estabilização. Se um nó é a raiz, então ele só atualiza seu número de descendentes até seu número total estar estável (Algoritmo 4). Os parâmetros *peFolha* e *peRaiz* são usados para definir o critério de estabilização em nós não raiz e na raiz, respectivamente. Uma vez que a fase de agregação está completa, a variável local da raiz *definiuDescendentes* é definida em verdadeiro, e o processo de alocação de endereços é iniciado pela raiz (Algoritmo 5).

```

1: maxTimerFolha = peFolha *  $DIO_{min}$ ;
2: timer = rand( $1/2 * DIO_{min}, DIO_{min}$ );
3: count = 0;
4: while NÃO- $DIO_{MHCL}$ -DE-PAI do
5:   if NÃO-RAIZ and ESTOUROU-TIMER then
6:     if definiuPai and ( $count < 1$ ) then
7:       envia  $DAO_{MHCL}$  para pai;
8:     end if
9:     if MUDOU-COUNT then
10:      envia  $DAO_{MHCL}$  para pai;
11:      reseta timer;
12:    else
13:      if timer < maxTimerFolha then
14:        timer *= 2;
15:      end if
16:    end if
17:  end if
18: end while

```

Algoritmo 3: MHCL bottom-up: Timer Agregação

```

1: definiuDescendentes = FALSE;
2: maxTimerRaiz = peRaiz *  $DIO_{min}$ ;
3: timer = rand( $1/2 * DIO_{min}, DIO_{min}$ );
4: count = 0;
5: while NÃO definiuDescendentes do
6:   if É-RAIZ and ESTOUROU-TIMER then
7:     if MUDOU-COUNT then
8:       reseta timer;
9:     else
10:      if timer < maxTimerRaiz then
11:        timer *= 2;
12:      else
13:        definiuDescendentes = TRUE;
14:      end if
15:    end if
16:  end if
17: end while

```

Algoritmo 4: MHCL bottom-up: Agregação (Raiz)

3.4. Tabelas de Roteamento e Encaminhamento

Cada nó, após o novo endereçamento, deve possuir uma tabela de roteamento com uma entrada para cada filho. Cada entrada possui duas informações: o endereço final da faixa de endereços do filho, e o endereço do filho. Dessa forma, para realizar o encaminhamento de mensagens basta comparar o endereço do destino com o último endereço da faixa de cada filho (Algoritmo 6). Como a inserção na tabela de roteamento é realizada de forma sequencial, os endereços estão em ordem crescente na tabela.

```

1: ESTÁVEL = FALSE;
2: if MHCL-top-down then
3:   ESTÁVEL = definiuFilhos;
4: else
5:   ESTÁVEL = definiuDescendentes or NÃO-RAIZ;
6: end if
7: if ESTÁVEL and (É-RAIZ or RECEBEU- $DIO_{MHCL}$ ) then
8:   divide endereços disponíveis;
9:   for cada filho  $c_i$  do
10:    envia  $DIO_{MHCL}$  para  $c_i$ ;
11:    if NÃO  $DIOACK_{MHCL}$  then
12:      envia  $DIO_{MHCL}$  para  $c_i$ ;
13:    end if
14:  end for
15: end if

```

Algoritmo 5: MHCL: Distribuição de endereços IPv6

```

1: i = 0;
2: while IPv6-destino > finalFaixaIP[i] do
3:   i++;
4: end while
5: encaminha para  $filhos[i]$ ;

```

Algoritmo 6: Encaminhamento de Mensagens

4. Resultados de Simulações

O protocolo MHCL apresenta características compatíveis com as necessidades de uma rede sem fio de baixa potência. Preparado para possuir um comportamento robusto em caso de falhas e com um algoritmo adaptado para as restrições da rede, ele se mostra uma boa alternativa para o endereçamento de nós em redes IPv6 *multihop*, auxiliando no roteamento e solucionando problemas em protocolos existentes. A fim de comprovar esses resultados, simulações foram realizadas.

Na Seção 4.1 são descritos os cenários e parâmetros de configuração das simulações. Na Seção 4.2 a fase inicialização da rede é avaliada de acordo com o tempo, número de mensagens enviadas e número de nós endereçados. Por fim, na Seção 4.3 são avaliadas as taxas de entrega das mensagens de aplicação.

4.1. Configurações da Simulação

O algoritmo MHCL foi implementado como uma rotina do protocolo RPL no sistema operacional *Contiki* [Dunkels et al. 2004]. Os resultados apresentam uma comparação entre o protocolo RPL com sua implementação padrão no *Contiki OS*, que utiliza uma configuração estática de endereços IPv6. Para realizar os experimentos, foi utilizado o Cooja [Eriksson et al. 2009], um simulador de redes sem fio de baixa potência com nós do tipo *SkyMote*, com 10KB de memória RAM, executando o sistema operacional *Contiki OS* com implementação do protocolo RPL [Dunkels et al. 2004].

Foram simuladas redes com 9, 25, 49 e 81 nós em uma distribuição regular, com raio de comunicação de 50 metros e 35 metros de distância entre cada nó. Para cada valor apresentado anteriormente, foram realizadas 8 simulações. Além disso, foram considerados modelos com e sem falhas na rede. As falhas simuladas foram de dois tipos: TX e RX. Segundo implementação do *Cooja*, em falhas do tipo TX a falha está na transmissão de pacotes, sendo que em $1 - TX\%$ de transmissões, nenhum dispositivo recebe o pacote enviado. Em falhas do tipo RX, em $1 - RX\%$ de transmissões de pacotes, apenas o nó destino não recebe o pacote, ou seja, é como se a falha ocorresse no link. Nos nossos experimentos, avaliamos a robustez do protocolo MHCL a cada tipo de falha separadamente. Portanto, foram aplicadas falhas com taxa de 95% não simultaneamente, ou seja, quando uma das variáveis está em 95% a outra, obrigatoriamente, está em 100%.

Na camada de aplicação, nós utilizamos como base um exemplo disponível no *Cooja* (rpl-udp), que utiliza o protocolo UDP na camada de transporte, ou seja, não implementa confirmação ou retransmissão de pacotes. Fizemos algumas modificações na aplicação, de forma que cada nó envie uma mensagem para a raiz e ela responda a cada uma dessas mensagens. Dessa forma, a aplicação envia $n - 1$ mensagens ascendentes e $n - 1$ descendentes. Essas mensagens começam a ser trocadas após 180 segundos de simulação. A simulação termina quando a última mensagem de aplicação é respondida, ou quando o tempo para esse evento ser completado se encerra, em caso de falhas. A Tabela 1 apresenta os parâmetros dos algoritmos apresentados na Seção 3, que são usados para estimar o tempo de estabilização da rede da seguinte forma: cada temporizador usado no protocolo MHCL começa com um valor mínimo, igual a $I = 2^{DIO_{min}}$ ms (um parâmetro usado pelo próprio RPL) e um valor máximo, igual a $pe * 2^{DIO_{min}}$, onde pe é o parâmetro de estabilização de cada rotina executada (veja os algoritmos apresentados na Seção 3).

Parâmetro	peFilho	pePai	peFolha	peRaiz	DIO_{min}
Valor	2	4	4	8	6

Tabela 1. Valor dos parâmetros de estabilização nos algoritmos

4.2. Inicialização da Rede

Para avaliar a inicialização da rede três parâmetros foram considerados. O primeiro deles, o tempo de inicialização, leva em consideração quanto tempo a rede precisa para se estruturar e obter as informações necessárias para realizar o roteamento. Os dois outros aspectos avaliados são a quantidade de mensagens do tipo DAO e DIO trocadas durante essa fase de inicialização.

4.2.1. Número de Mensagens

Para avaliar a quantidade de mensagens enviadas por cada protocolo, os números de mensagens DAO e DIO foram contados separadamente. A Figura 4 apresenta os resultados obtidos. Como esperado, o número de DIOs enviados não se difere estatisticamente entre os algoritmos, com uma confiança de 95% (Figura 3(b)). Isto porque o mecanismo de descoberta e manutenção do DAG não foi alterado da implementação do RPL para o MHCL. Para o endereçamento dos nós no MHCL são utilizadas mensagens do tipo DIO, entretanto o número de mensagens adicionais é muito pequeno se comparado com a quantidade total de mensagens DIO transmitidas na rede. Entretanto, como o mecanismo de descoberta de rotas, que utiliza mensagens DAO, é diferente no RPL e MHCL, o número de mensagens desse tipo enviadas tem uma diferença significativa (Figura 3(a)). Com 95% de confiança podemos afirmar que o MHCL envia menos mensagens do tipo DAO do que o RPL, em média $10\times$ menos mensagens. Isto porque no MHCL existe um algoritmo de coleta de informações sobre a topologia que, uma vez encerrado, não envia mais mensagens DAO. O RPL realiza a notificação de rotas durante o funcionamento da rede, o que faz com que mensagens DAO sejam transmitidas continuamente para notificação de novas rotas.

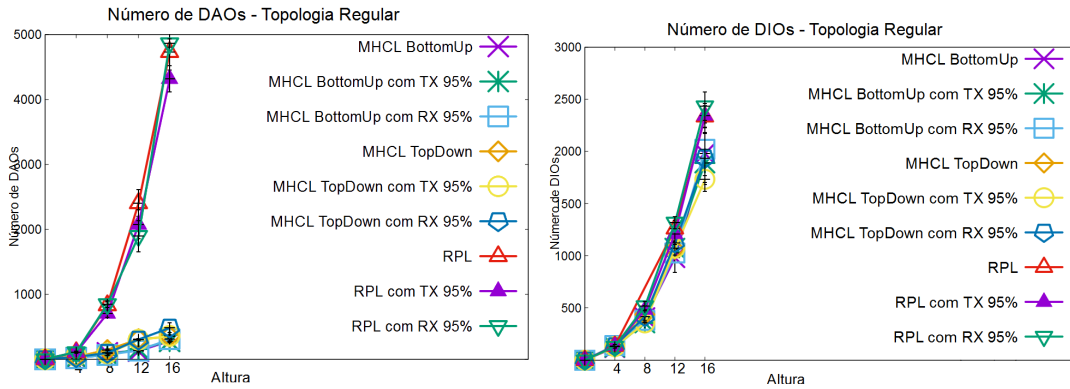
4.2.2. Tempo de Inicialização

Para o algoritmo MHCL foi considerado como tempo de inicialização o tempo necessário para todos os nós serem endereçados. No RPL foi medido o tempo decorrido até a raiz ter conhecimento de $n - 1$ rotas, ou seja, conhecer uma rota para cada nó da rede. A Figura 4(a) apresenta o gráfico com o tempo de inicialização de cada protocolo. Podemos observar que o tempo de inicialização tem um crescimento linear com o número de nós da rede, para todos os algoritmos simulados. No entanto, se considerarmos um intervalo de confiança de 95%, não é possível afirmar que um algoritmo é mais rápido do que o outro, pois existe a sobreposição dos intervalos de confiança. Note que estamos trabalhando com algoritmos assíncronos e a inicialização de cada nó é aleatória, o que é capaz de alterar significativamente o tempo de execução. Para isto, foi realizado o *Teste t*. Na comparação das estratégias bottom-up e top-down o intervalo de confiança resultante foi (-44.578,74.442), para a comparação entre o RPL e o top-down (-66.81,41.21) e, por fim, entre o RPL e a estratégia bottom-up (-97.022,41.578). Com esses resultados podemos

concluir que com 95% de confiança, o tempo dos três algoritmos é estatisticamente igual, uma vez que os intervalos de confiança incluem o valor zero, para 81 nós.

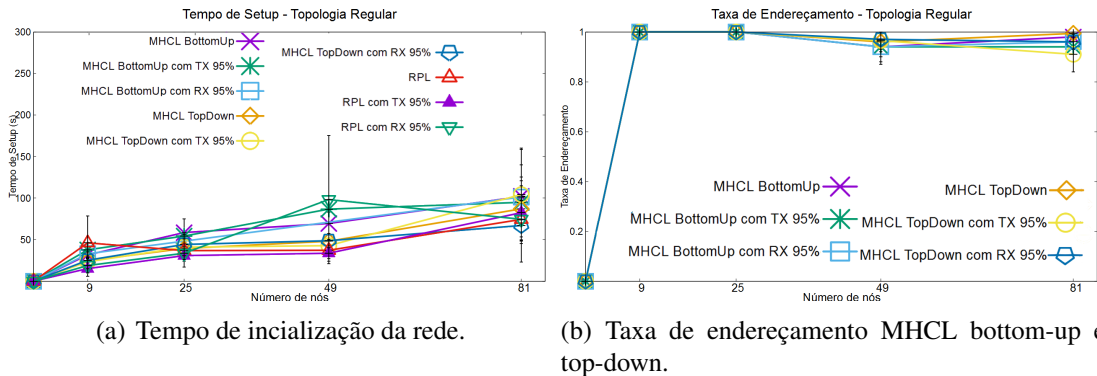
4.2.3. Taxa de Endereçamento

O endereçamento dos nós pelo MHCL é feito durante a inicialização da rede. Como a rede pode apresentar colisões e falhas nos nós e nos links, algumas mensagens de endereçamento (do tipo DIO_{MHCL}) podem ser perdidas, mesmo usando mensagens de confirmação (do tipo $DIOACK_{MHCL}$), e realizando tentativas de retransmissão (nas simulações apresentadas, até 3 vezes). A Figura 4(b) apresenta os gráficos que compara a taxa de endereçamento de nós das estratégias bottom-up e top-down. Podemos observar que, apesar de colisões e falhas intermitentes de nós e links, a taxa de endereçamento se mantém entre 90% e 100% em ambas as estratégias. Como é possível observar na Figura 4(b), existe a sobreposição dos intervalos de confiança e ambos contêm a média da outra estratégia. Portanto, podemos afirmar que, estatisticamente, o endereçamento de ambas as estratégias apresenta uma mesma taxa de sucesso.



(a) Número de DAOs em função da altura do DAG. (b) Número de DIOs em função da altura do DAG.

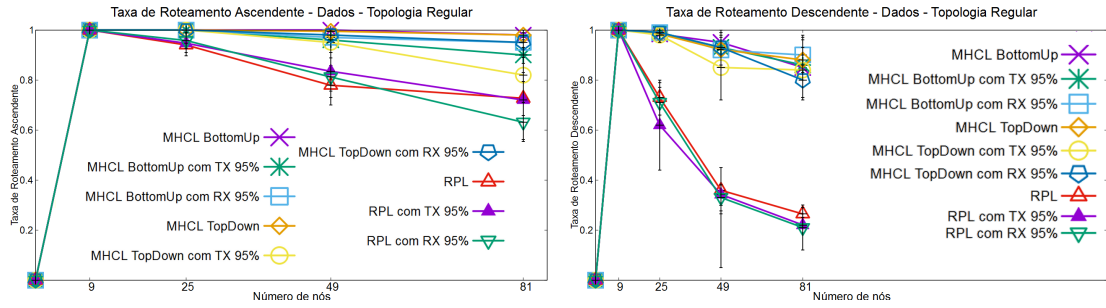
Figura 3. Número de mensagens na inicialização da rede.



(a) Tempo de inicialização da rede.

(b) Taxa de endereçamento MHCL bottom-up e top-down.

Figura 4. Tempo de inicialização e taxa de endereçamento.



(a) Taxa de entrega de mensagens de aplicação ascendentes. (b) Taxa de entrega de mensagens de aplicação descendentes.

Figura 5. Taxa de entrega de mensagens de aplicação.

4.3. Mensagens de Aplicação

Por fim, a taxa de entrega de mensagens ascendentes e descendentes foi calculada (Figura 5). Note que a camada de aplicação simulada não realiza confirmações e retransmissões de mensagens, portanto, uma colisão ou falha em um link ou nó pode causar perdas definitivas de mensagens. Para o caso de mensagens ascendentes (Figura 5(a)), como o algoritmo de encaminhamento não foi alterado, as estratégias apresentam valores próximos. Entretanto, como durante o algoritmo de aplicação o RPL ainda envia mensagens do tipo DAO, essas mensagens disputam o meio com as mensagens ascendentes da aplicação, aumentando chance de colisões. No MHCL a transmissão de DAO após a fase de inicialização não ocorre, resultando em taxa de sucesso maior.

Para o roteamento descendente (Figura 5(b)) a diferença de desempenho entre MHCL e RPL é visível. Como é possível observar, enquanto o RPL, para 81 nós, apresenta uma taxa de entrega de 26% em média, o MHCL bottom-up e top-down obtêm sucesso em cerca de 87% das mensagens descendentes. Com 95% de confiança, podemos afirmar que o MHCL possui uma taxa de entrega de mensagens descendentes $3\times$ superior ao RPL, e as estratégias bottom-up e top-down do MHCL são equivalentes, em relação a essa taxa. Isto ocorre pois, no RPL, os nós não têm memória suficiente para armazenar as tabelas de roteamento necessárias para endereçar todos os seus descendentes, impossibilitando assim o roteamento de todas as mensagens, levando a uma baixa taxa de roteamento descendente.

5. Trabalhos Relacionados

O protocolo RPL, desde sua criação, foi muito estudado e, por isso, existem diversos trabalhos avaliando sua especificação e funcionamento. Em [Clausen et al. 2011] é realizada uma avaliação crítica do RPL, cerca de dois anos após sua criação. A falta de especificação de algumas partes do protocolo é um problema apontado, que conta com diversos pontos ainda não bem definidos do protocolo. Enquanto as mensagens do tipo DIO utilizam o Trickle para especificar a temporização de mensagens de maneira simples e de fácil entendimento, detalhes sobre mensagens do tipo DAO são obscuros e nenhum temporizador para essas mensagens é definido. Em [Ko et al. 2011] e [Zhang and Li 2014] o protocolo RPL é avaliado de acordo com sua implementação em cada um dos sistemas operacionais. No primeiro trabalho, [Ko et al. 2011], é apresentado uma implementação do RPL, chamada *TinyRPL*. Em seu trabalho, eles usam a pilha IP de baixa potência de

Berkley (BLIP) no *TinyOs* [Levis et al. 2005], que interage com sua implementação do protocolo. A implementação do *TinyRPL* conta com todos os mecanismos básicos da definição do RPL, enquanto omite todos os opcionais. Para a validação, o protocolo RPL é comparado com o *Collection Tree Protocol* (CTP) [Gnawali et al. 2009], o protocolo padrão para coleta de dados do *TinyOs*. Um dos benefícios do RPL, em comparação com o CTP, é a possibilidade de tipos variados de padrão de tráfego (P2P, MP2P e P2MP), além de sua capacidade de conectar nós à Internet diretamente, trocando pacotes com endereços IPv6 global. Em [Zhang and Li 2014] é estudada a performance do protocolo RPL no sistema operacional Contiki OS [Dunkels et al. 2004]. O algoritmo é simulado a fim de mostrar na prática implicações baseadas na descrição do protocolo. Por exemplo, de acordo com o mecanismo de inicialização da rede, é possível imaginar que mensagens de controle podem inundar a rede durante a construção do DAG e que essa etapa pode levar muito tempo. Os resultados mostram que inicialmente o número de mensagens de controle é superior ao número de mensagem de dados, mas essas mensagens tendem a diminuir após cerca de 60 segundos, o que pode ser determinado como o tempo de inicialização da rede, para os casos avaliados.

6. Conclusões

Neste trabalho nós propusemos o MHCL, uma estratégia de alocação de endereços IPv6 para redes sem fio *multihop* de baixa potência. As principais características do MHCL são: (1) Eficiência de memória: ao utilizar a topologia acíclica da rede para realizar um particionamento hierárquico do espaço de endereçamento disponível para um roteador de borda, os endereços de nós pertencentes a uma mesma sub-árvore são agrupados em uma única entrada na tabela de roteamento de um nó, tornando o tamanho da tabela $O(k)$, onde k é o número de filhos diretos do nó; isso se contrapõe ao tamanho $O(n)$ das tabelas de roteamento do RPL, onde n é o número total de descendentes de cada nó; (2) Eficiência temporal: as principais rotinas do MHCL são baseadas em temporizadores que rapidamente se adaptam à dinâmica da rede, possibilitando uma rápida fase de configuração de nós; (3) Eficiência em número de mensagens: o número de mensagens enviadas é adaptado à dinâmica da rede, através dos mesmos temporizadores acima; (4) Eficiência de endereçamento: o MHCL consegue endereçar perto de 100% dos nós, mesmo em cenários com falha; (5) Eficiência na entrega de mensagens de aplicação: o roteamento descendente do MHCL se mostrou eficaz, entregando até $3\times$ mais mensagens do que o RPL. Como trabalhos futuros, pretendemos analisar a robustez do MHCL a mudanças na topologia da rede que se estendam além de falhas intermitentes de nós e links individuais.

Referências

- Contiki: The open source os for the internet of things. <http://www.contiki-os.org/>.
- Clausen, T., Herberg, U., and Philipp, M. (2011). A critical evaluation of the ipv6 routing protocol for low power and lossy networks (rpl). In *Wireless and Mobile Computing, Networking and Communications (WiMob), 2011 IEEE 7th International Conference on*, pages 365–372.
- Dunkels, A., Gronvall, B., and Voigt, T. (2004). Contiki - a lightweight and flexible operating system for tiny networked sensors. In *Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks, LCN '04*, pages 455–462, Washington, DC, USA. IEEE Computer Society.
- Eriksson, J., Österlind, F., Finne, N., Tsiftes, N., Dunkels, A., Voigt, T., Sauter, R., and Marrón, P. J. (2009). Cooja/mspsim: Interoperability testing for wireless sensor networks. In *Proceedings of the 2Nd International Conference on Simulation Tools and Techniques, Simutools '09*, pages 27:1–27:7, ICST, Brussels,

- Belgium, Belgium. ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering).
- Gnawali, O., Fonseca, R., Jamieson, K., Moss, D., and Levis, P. (2009). Collection tree protocol. In *Proceedings of the 7th ACM Conference on Embedded Networked Sensor Systems, SenSys '09*, pages 1–14, New York, NY, USA. ACM.
- Ko, J., Dawson-Haggerty, S., Gnawali, O., Culler, D., and Terzis, A. (2011). Evaluating the Performance of RPL and 6LoWPAN in TinyOS. In *Proceedings of Extending the Internet to Low power and Lossy Networks (IPSN 2011)*.
- Levis, P., Brewer, E., Culler, D., Gay, D., Madden, S., Patel, N., Polastre, J., Shenker, S., Szewczyk, R., and Woo, A. (2008). The emergence of a networking primitive in wireless sensor networks. *Commun. ACM*, 51(7):99–106.
- Levis, P., Madden, S., Polastre, J., Szewczyk, R., Whitehouse, K., Woo, A., Gay, D., Hill, J., Welsh, M., Brewer, E., and Culler, D. (2005). TinyOS: An Operating System for Sensor Networks. In Weber, W., Rabaey, J., and Aarts, E., editors, *Ambient Intelligence*, chapter 7, pages 115–148. Springer Berlin Heidelberg, Berlin/Heidelberg.
- Levis, P., Patel, N., Culler, D., and Shenker, S. (2004). Trickle: A self-regulating algorithm for code propagation and maintenance in wireless sensor networks. In *Proceedings of the 1st Conference on Symposium on Networked Systems Design and Implementation - Volume 1, NSDI'04*, pages 2–2, Berkeley, CA, USA. USENIX Association.
- Liu, Y., He, Y., Li, M., Wang, J., Liu, K., and Li, X.-Y. (2013). Does wireless sensor network scale? a measurement study on greenorbs. *IEEE Trans. Parallel Distrib. Syst.*, 24(10):1983–1993.
- P. Levis, T. Clausen, J. H. O. G. J. K. (2011). The trickle algorithm. <https://tools.ietf.org/html/draft-ietf-roll-trickle-08>.
- Tolle, G., Polastre, J., Szewczyk, R., Culler, D., Turner, N., Tu, K., Burgess, S., Dawson, T., Buonadonna, P., Gay, D., and Hong, W. (2005). A macroscope in the redwoods. In *Proceedings of the 3rd International Conference on Embedded Networked Sensor Systems, SenSys '05*, pages 51–63, New York, NY, USA. ACM.
- Upadhyayula, S., Annamalai, V., and Gupta, S. (2003). A low-latency and energy-efficient algorithm for convergecast in wireless sensor networks. In *Global Telecommunications Conference, 2003. GLOBE-COM '03. IEEE*, volume 6, pages 3525–3530 vol.6.
- Vasseur, J., Agarwal, N., Hui, J., Shelby, Z., Bertrand, P., and Chauvenet, C. (2011). Rpl: The ip routing protocol designed for low power and lossy networks. Internet Protocol for Smart Objects (IPSO) Alliance.
- Vasseur, J.-P. and Dunkels, A. (2010). *Interconnecting Smart Objects with IP: The Next Internet*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- Vicaire, P., He, T., Cao, Q., Yan, T., Zhou, G., Gu, L., Luo, L., Stoleru, R., Stankovic, J. A., and Abdelzaher, T. F. (2009). Achieving long-term surveillance in vigilnet. *ACM Trans. Sen. Netw.*, 5(1):9:1–9:39.
- Werner-Allen, G., Lorincz, K., Johnson, J., Lees, J., and Welsh, M. (2006). Fidelity and yield in a volcano monitoring sensor network. In *Proceedings of the 7th Symposium on Operating Systems Design and Implementation, OSDI '06*, pages 381–396, Berkeley, CA, USA. USENIX Association.
- Winter, T., Thubert, P., Brandt, A., Hui, J., Kelsey, R., Levis, P., Pister, K., Struik, R., Vasseur, J., and Alexander, R. (2012). RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks. RFC 6550 (Proposed Standard).
- Zhang, T. and Li, X. (2014). Evaluating and analyzing the performance of rpl in contiki. In *Proceedings of the First International Workshop on Mobile Sensing, Computing and Communication, MSCC '14*, pages 19–24, New York, NY, USA. ACM.